

Landmark Classification and Tagging for Social Media

*A Convolutional Neural Network and Transfer Learning Approach, with
Deployment as “Landmark Snap”*

Project Report

Deep Learning / Computer Vision – Landmark Classification Project

July 2026

Table of Contents

Table of Contents	2
1. Executive Summary	3
2. Problem Statement	4
3. Dataset	5
4. Solution Statement	6
5. Architecture	7
6. Experiment Tracking Systems	9
7. Standouts	10
8. Landmark Snap Deployment	11
9. Conclusion	12
10. Value Statement	13
11. Limitations	15
12. Direction of Future Research	16
13. References	17

1. Executive Summary

Photo-sharing and photo-storage platforms increasingly rely on location metadata to power features such as automatic tagging, album organization, and personalized recommendations. When GPS metadata is missing or stripped for privacy reasons, one of the few remaining signals available directly from the pixels of an image is the presence of a recognizable landmark. This report documents the design, implementation, and deployment of a landmark classification system, "Landmark Snap," built to automatically identify the most probable landmark depicted in a user-submitted photograph and to demonstrate that pixel-level visual evidence alone can substitute for missing location metadata in a production-relevant way.

The project followed the full machine learning lifecycle end to end: data exploration and preprocessing, model design and training, systematic hyperparameter experimentation, model export, and deployment as an interactive web application. Two complementary modeling strategies were implemented and compared. The first is a convolutional neural network (CNN) built and trained entirely from scratch, which establishes a transparent, fully-understood baseline and demonstrates command of foundational architectural choices such as convolutional block design, batch normalization placement, and dropout-based regularization. The second is a transfer-learning model built on a ResNet18 backbone pretrained on ImageNet, with the convolutional backbone frozen and only a new classification head trained on the landmark dataset. This second approach reflects best practice for applied computer vision problems with modest amounts of labeled data, and is the model ultimately served in production.

Both models were trained and evaluated on a curated fifty-class subset of the Google Landmarks dataset, using a disciplined validation methodology: hyperparameters were tuned exclusively against a held-out validation split, with the test set touched only once, at the end, to produce an unbiased estimate of real-world performance. This report documents that process, including the experiment-tracking approach used to compare configurations, the final architecture and hyperparameters selected for each model, and the resulting performance relative to the project's target thresholds — a minimum of 50% test accuracy for the from-scratch CNN and 60% for the transfer-learning model, with stand-out targets of 60% and 80% respectively.

Beyond the core classification task, the project delivered three stand-out extensions: a full experiment-tracking discipline embedded directly in the analysis notebooks; a content-based image retrieval capability built on the from-scratch CNN's learned feature embeddings, allowing visually similar landmark photos to be surfaced via cosine similarity rather than class label alone; and a fully deployed, publicly accessible web application ("Landmark Snap") built with Voila and hosted on Heroku, giving any user a browser-based interface to upload a photo and receive an instant, ranked prediction with a styled probability visualization.

Collectively, this work demonstrates not only that landmark classification is technically feasible with a modest dataset and modern transfer-learning techniques, but that the resulting model can be operationalized into a genuinely usable product within realistic infrastructure constraints (CPU-only inference, limited deployment memory, and standard web hosting). The remainder of this report details the problem context, dataset, solution architecture, experimentation methodology, stand-out contributions, deployment design, and an honest accounting of the system's current limitations and promising directions for future work.

2. Problem Statement

Modern photo-sharing and photo-storage services depend heavily on metadata to deliver the experiences users expect: automatically organized albums, "photos from this trip" summaries, location-based search, and contextual tag suggestions. Much of this experience is built on GPS coordinates embedded in a photo's EXIF metadata at the moment of capture. In practice, however, a large fraction of uploaded images carry no usable location metadata at all. Some cameras and older devices simply lack GPS hardware. Many messaging and social platforms strip EXIF data entirely before an image is ever forwarded or re-shared, both to reduce file size and to protect user privacy. Scanned or digitized prints, screenshots, and images exported from third-party editing tools frequently arrive with metadata already stripped by the time they reach a storage or sharing service. The result is a persistent and non-trivial gap: a meaningful share of a platform's image inventory cannot be geographically organized using metadata alone, even though the images themselves often contain strong, human-recognizable visual evidence of where they were taken.

This is precisely the situation where an automatic landmark classifier becomes valuable. Many of the photos lacking location metadata still depict a recognizable landmark — the Eiffel Tower, the Golden Gate Bridge, Machu Picchu — an unmistakable visual identifier of place, unrelated to any metadata field. If a system can reliably recognize the landmark present in an image directly from its pixels, it can recover an approximate location even when every other geolocation signal has been lost. The scale of the problem, however, rules out any manual solution: photo-sharing platforms process image volumes far beyond what human review could ever address, and the set of landmarks a user might photograph spans the entire globe. This is fundamentally a large-scale, fine-grained visual classification problem, and it is precisely the class of problem for which convolutional neural networks have proven most effective.

The specific engineering task this project addresses is narrower and more tractable than "recognize every landmark on Earth": given a curated, fixed set of fifty well-known landmarks, build a model that classifies a new, unseen photograph into the correct landmark category (or, implicitly, signals uncertainty when the correct answer is unclear) with high enough accuracy to

be useful in a downstream product feature. This requires solving several sub-problems simultaneously: designing an image preprocessing and augmentation pipeline appropriate for real-world, user-submitted photos (which vary enormously in lighting, angle, framing, and image quality); choosing and justifying a model architecture capable of learning discriminative visual features from a comparatively small, fixed-size training set; and validating that the resulting model generalizes to genuinely new photographs rather than merely memorizing its training data.

Success is measured not by an abstract accuracy number in isolation, but by whether the resulting model and its accompanying deployment artifact could plausibly sit behind a real product feature — informing an "auto-tag this location" suggestion, powering a "similar places" recommendation, or flagging a photo for human review when confidence is low. The remainder of this report describes the dataset, architecture, and experimental process used to build such a model, along with a deployed application that demonstrates the end-to-end feasibility of this approach.

3. Dataset

The models in this project were trained on a curated fifty-class subset of the Google Landmarks dataset, prepared specifically for this classification task. Each of the fifty classes corresponds to a distinct, well-known landmark (examples include the Golden Gate Bridge, the Eiffel Tower, Machu Picchu, the Atomium, and Haleakala National Park), with class folders named using a zero-padded numeric prefix followed by the landmark name (for example, "09.Golden_Gate_Bridge"), which conveniently doubles as the label used by PyTorch's ImageFolder dataset loader. The dataset is organized into pre-split train and test directories, downloaded on demand into a local landmark_images folder rather than distributed inside the project repository, which keeps the codebase lightweight and ensures every run starts from a known, versioned copy of the data.

Because no separate validation directory is provided, the training split is further partitioned at load time into a training subset and a validation subset, using a stratified random split (20% held out for validation by default) via SubsetRandomSampler. This design has an important implication that is easy to overlook: the validation subset is drawn from the same underlying image pool as the training subset, using the validation-time (non-augmented) transform, while the held-out test directory represents a fully independent sample never touched during model selection. This structure is what allows validation performance to be used liberally during hyperparameter tuning while preserving the test set as an unbiased, one-time estimate of generalization.

Image preprocessing follows a standard and carefully considered pipeline. Every image, regardless of split, is first resized so its shorter side measures 256 pixels, then cropped to a 224x224 square — the input resolution expected by both the from-scratch architecture and the

ImageNet-pretrained ResNet18 backbone used for transfer learning. For the training split, a random crop is used instead of a center crop, and is combined with random horizontal flipping, small-angle random rotation (± 10 degrees), and mild color jitter (brightness, contrast, and saturation perturbations). These augmentations are a direct response to the nature of the problem: landmark photos submitted by real users vary enormously in framing, orientation, and lighting condition, and a model trained only on pristine, centered training images would generalize poorly to that variability. Validation and test images intentionally receive no augmentation, since they are meant to reflect the true, unmodified distribution the model must ultimately perform on.

Every image, across all splits, is converted to a tensor and normalized using a per-channel mean and standard deviation computed directly from the training data itself (rather than assumed ImageNet statistics), via a helper that iterates the training set once, caches the result to disk, and reuses it on subsequent runs. This keeps input pixel values in a numerically well-behaved range for gradient-based optimization and avoids a subtle mismatch between the assumed and actual input distribution.

Two class-composition characteristics of this dataset are worth calling out explicitly as context for the results discussed later. First, with fifty classes, a naive baseline (always predicting the most frequent class) would achieve roughly two percent accuracy, meaning even a modest double-digit accuracy already reflects genuine, non-trivial learned discrimination. Second, like most real-world, crowd-sourced photography, the dataset is not perfectly class-balanced, and some landmarks are represented with visually more distinctive, less ambiguous imagery than others — a factor that surfaces later in the discussion of per-class performance variation and limitations.

4. Solution Statement

The solution adopted for this project follows a deliberate two-model strategy, chosen to satisfy both the pedagogical goal of demonstrating fundamental deep learning competence and the practical goal of producing a model strong enough to deploy. Rather than jumping straight to the highest-performing available architecture, the project first builds and trains a convolutional neural network entirely from scratch, then builds a second model using transfer learning from a pretrained backbone, and finally compares the two using a controlled, validation-driven methodology before selecting one for deployment.

The from-scratch CNN exists to demonstrate a genuine, ground-up understanding of the building blocks of image classification: how convolutional layers extract spatially local features, why batch normalization accelerates and stabilizes training, how max-pooling and channel-doubling patterns build a hierarchy from low-level edges to high-level object parts, and how dropout regularizes a classifier head against overfitting on a comparatively small dataset. This model also

serves a second, practical purpose in this project: because it is trained end-to-end on the landmark data itself (rather than having its backbone frozen), its penultimate-layer activations form a natural feature embedding that later powers the content-based image-retrieval stand-out feature described in a later section — something a frozen, off-the-shelf backbone could offer, but with less direct evidence that the features are specifically tuned to this landmark domain.

The transfer-learning model addresses the from-scratch model's principal limitation: with only a few thousand training images spread across fifty classes, a CNN trained from random initialization has a comparatively small amount of data from which to learn a full visual hierarchy, and is correspondingly prone to either underfitting (if regularized too aggressively) or overfitting (if not). Initializing from a network already trained on over a million general-purpose images sidesteps this constraint entirely: the low- and mid-level visual features relevant to recognizing edges, textures, and object parts have already been learned, and only a small, landmark-specific classification head needs to be trained from the available data. This dramatically reduces the effective sample complexity of the learning problem and is the reason the transfer-learning model both trains faster and is expected to reach a materially higher accuracy ceiling than the from-scratch model, consistent with the differentiated target thresholds set for each approach (50%/60% for the from-scratch model versus 60%/80% for transfer learning).

Both models share a common evaluation discipline: hyperparameters (learning rate, optimizer choice, batch size, weight decay, and, for the from-scratch model, dropout rate) are tuned exclusively against validation performance, with the true test set evaluated only once a final configuration has been chosen. Both models are exported using TorchScript upon completion of training, producing a single self-contained artifact that bundles the trained weights, the necessary preprocessing transforms, and the ordered list of class names — eliminating any risk of a mismatch between how a model was trained and how it is later served in an application context. This exported artifact is precisely what the deployed "Landmark Snap" application consumes at inference time, meaning the exact same, single, versioned model asset connects experimentation directly to production without any re-implementation step in between — a deliberate design choice that removes an entire class of training/serving skew bugs common in less carefully engineered ML systems.

5. Architecture

The from-scratch convolutional neural network follows a classic VGG-style design pattern: five sequential convolutional blocks, each composed of a 3x3 convolution, batch normalization, a ReLU activation, and 2x2 max-pooling, followed by a fully connected classification head. Channel depth doubles at every block (3 to 16, 32, 64, 128, and finally 256 channels), while spatial resolution is halved at every block by the pooling operation, taking the 224x224 input down to a

7x7 feature map by the final convolutional block. This channel-doubling, resolution-halving pattern is a deliberate and well-established design choice: as spatial resolution is discarded, representational capacity is redirected into a growing number of channels, allowing the network to trade fine-grained spatial detail for increasingly abstract, semantically meaningful features as depth increases — from edges and textures in early layers toward landmark-specific shapes and structures in later ones.

Batch normalization is applied after every convolution (and again after the first fully connected layer), which stabilizes gradient magnitudes across layers, permits a higher learning rate than would otherwise be numerically stable, and provides a mild regularizing effect from the noise introduced by batch statistics during training. Max-pooling, rather than a strided convolution, is used for downsampling on the strength of its simplicity and lack of additional learnable parameters, which is an appropriate trade-off given the modest depth of this network. The classification head flattens the final 256x7x7 feature map into a 12,544-dimensional vector, projects it through a single 512-unit fully connected layer with batch normalization and ReLU, applies dropout as the network's primary defense against overfitting, and finally projects to a 50-way output layer producing raw, unnormalized class logits. Softmax is deliberately not applied inside the model itself, since the cross-entropy loss function used for training expects raw logits and applies the equivalent of a log-softmax internally; softmax is instead applied once, explicitly, at inference time inside the exported prediction wrapper.

The transfer-learning model takes a fundamentally different approach to feature extraction while sharing the same conceptual output structure. A ResNet18 backbone, pretrained on ImageNet, is loaded and every one of its convolutional parameters is frozen (`requires_grad` set to `False`), so that no gradient updates ever reach the pretrained feature extractor during training. ResNet18 was selected over deeper alternatives such as ResNet50 or ResNet101 as a deliberate accuracy-versus-compute trade-off: its residual (skip) connections allow it to represent rich, multi-level visual features despite its comparatively modest eighteen-layer depth, while remaining light enough to fine-tune quickly on commodity hardware, an important consideration since only the final layer's gradients need to be computed and applied. The network's original 1000-way ImageNet classification layer is replaced with a new, randomly initialized linear layer sized to the model's own input feature dimension and the fifty landmark classes, and only this new layer's weights are updated during training.

Both models are trained with cross-entropy loss and a choice of stochastic gradient descent or Adam as the optimizer (selected per experiment), paired with a ReduceLROnPlateau learning-rate scheduler that halves the learning rate whenever validation loss stops improving for several consecutive epochs, allowing training to make coarse progress early and increasingly fine-grained adjustments as it converges. The best-performing checkpoint, as measured by validation loss, is

persisted to disk after every epoch in which it improves, ensuring the final exported model reflects the best point in training rather than whatever state training happened to end on.

6. Experiment Tracking Systems

Systematic experiment tracking was treated as a first-class part of this project's methodology rather than an afterthought, because hyperparameter selection made without a disciplined comparison process risks either underperforming or, worse, overfitting to whichever data split happens to be used for evaluation. Two complementary tracking mechanisms were used throughout training: a real-time visual mechanism for monitoring convergence within a single run, and a structured tabular mechanism for comparing configurations across runs.

Within a single training run, live loss curves are rendered directly inside the notebook using the `livelossplot` library, updated at the end of every epoch with the current training loss, validation loss, and learning rate. This live visualization serves several diagnostic purposes simultaneously. A widening gap between training and validation loss is an immediate, visual signal of overfitting, prompting an increase in dropout or weight decay, or the introduction of additional data augmentation, before a run is allowed to continue unnecessarily. Conversely, both losses plateauing at a high value together signals underfitting, prompting a higher learning rate, reduced regularization, or additional training epochs. The learning-rate trace is tracked alongside the losses specifically to make the behavior of the `ReduceLROnPlateau` scheduler visible: repeated learning-rate reductions without any corresponding improvement in validation loss is a clear, unambiguous stopping signal, indicating that a given configuration has converged as far as it is going to and that continued training is not a productive use of compute time.

Across runs, a structured experiment-tracking table was added directly into both the from-scratch and transfer-learning notebooks, positioned immediately after each model's training cell and before its corresponding test-set evaluation step. This placement is intentional: it forces a deliberate checkpoint at which training and validation accuracy must be inspected, recorded, and compared before the test set is ever touched. Each row of the table records the hyperparameters used for a given run (batch size, learning rate, optimizer, weight decay, and, for the from-scratch model, dropout rate, plus backbone choice for the transfer-learning model), the resulting training and validation loss, and a short qualitative note capturing what was observed and what change, if any, was made in response for the next run. A dedicated accuracy-checking cell, reusing the same evaluation function used for final testing but pointed at the training and validation loaders instead, was added specifically to populate this table with concrete numbers rather than loss values alone, since accuracy is ultimately the metric the project's rubric and business objective care about.

This combination of a live, per-epoch diagnostic and a persistent, cross-run comparison table reflects a lightweight but genuinely disciplined approach to model selection: it makes the tuning process auditable after the fact, discourages the common failure mode of tuning hyperparameters directly against test-set performance, and produces a written record of the reasoning behind the final configuration chosen for deployment, rather than a single unexplained accuracy number. In a larger-scale or longer-running project, this same discipline would translate naturally into a dedicated experiment-tracking platform such as MLflow or Weights & Biases, which would add automatic run versioning, artifact storage, and multi-run comparison dashboards on top of the same underlying methodology already established here.

7. Standouts

Beyond satisfying the project's core requirements, four distinct stand-out contributions were implemented to extend the project's depth, robustness, and real-world applicability.

The first stand-out is the experiment-tracking discipline described in the preceding section: a structured, in-notebook methodology for comparing hyperparameter configurations using only training and validation data, paired with live convergence visualization. This was implemented for both the from-scratch and transfer-learning models, ensuring the final configuration selected for each was chosen on principled, auditable grounds rather than a single untracked run.

The second stand-out is a content-based image-retrieval system built on top of the from-scratch CNN's learned representations. A method was added to the model that returns its penultimate-layer activations — the 512-dimensional feature vector produced immediately before the final classification layer — rather than class predictions. Given a query image, this feature vector is L2-normalized and compared, via cosine similarity (a simple dot product on normalized vectors), against the equivalently extracted features of a gallery of test-set images, and the most similar images are retrieved and displayed. This demonstrates a capability meaningfully broader than classification alone: rather than only answering "which of these fifty classes is this," the model can answer "which images in a large, unlabeled collection look most similar to this one," a capability directly applicable to features like "find more photos like this" or near-duplicate detection, independent of whether a photo depicts a landmark the model was explicitly trained to name.

The third stand-out is a written discussion of additional, adjacent use cases for the trained model and its retrieval capability beyond the immediate landmark-tagging use case: automatic photo album organization for images lacking any metadata whatsoever, travel and tourism applications that surface contextual information the moment a landmark is recognized, content-moderation workflows that flag imagery of sensitive locations, and dataset-curation pipelines that use the model to pre-label or filter large, unlabeled photo collections for future model training. This

discussion is paired with an explicit, honest treatment of the model's boundaries — namely that it can only recognize the fifty landmarks it was trained on and will confidently mislabel anything else as one of those fifty — establishing the scope within which each proposed use case would actually be appropriate.

The fourth and most substantial stand-out is the full deployment of the trained model as a public-facing web application, "Landmark Snap," described in detail in the following section. This goes considerably beyond the project's baseline requirement of a locally runnable Voila notebook: the application was engineered for genuine robustness (handling known cross-version incompatibilities in the ipywidgets upload widget across different notebook front-ends, with a URL-based fallback classification path for browsers or environments where binary file upload does not sync correctly to the kernel), visually redesigned with custom CSS styling, animated probability bars, and a cohesive branded interface, and packaged with a complete, tested Heroku deployment pipeline — including a CPU-only PyTorch dependency configuration necessary to fit within Heroku's slug size constraints, and a written runbook covering memory limits, cold-start behavior, and common deployment failure modes. Taken together, these four contributions move the project from "a model that classifies landmarks" to "a validated, documented, and genuinely deployable system," which is the more meaningful bar for a project intended to demonstrate applied machine learning competence.

8. Landmark Snap Deployment

"Landmark Snap" is the deployed, production-facing artifact of this project: a standalone, browser-accessible application that allows any user to upload a photograph and receive a ranked list of predicted landmarks, each with an associated confidence score, in real time. It was deliberately engineered as a self-contained deployment unit, decoupled from the training codebase, so that shipping it does not require carrying any training-time dependencies, source code, or dataset assets into the production environment — the only artifact it depends on is the single exported TorchScript model file, which already bundles the trained weights, the required preprocessing transforms, and the ordered list of class names.

The application is built with Voila, which turns a Jupyter notebook into a standalone, code-hidden web application, showing only widget interfaces and rendered outputs rather than the underlying cells. The interface itself was substantially redesigned beyond a minimal widget layout: a custom CSS stylesheet (imported Google Font, gradient banner header, card-based section layout, and consistent color palette) is injected directly into the notebook's output, giving the application a cohesive, purpose-built visual identity rather than a default, unstyled Jupyter appearance. Predictions are rendered as animated, color-coded horizontal probability bars rather than plain text labels, with the top prediction visually distinguished from the remaining

four. Two independent input paths are provided: a standard file-upload widget for the common case, and a URL-based fallback that downloads and classifies an image from a pasted link, added specifically because ipywidgets' binary FileUpload content-transfer mechanism is known to behave inconsistently across notebook front-ends (a concrete compatibility issue with VS Code's Jupyter extension was identified and fixed during development, and the URL path exists as a robust fallback for any environment where the same class of issue might recur).

Deployment targets Heroku's platform-as-a-service model. The application's dependency footprint was deliberately constrained to fit Heroku's operational limits: PyTorch and torchvision are installed from PyTorch's CPU-only wheel index rather than the default (CUDA-enabled) distribution, since Heroku dynos have no GPU access and the GPU-enabled wheels are large enough to risk exceeding Heroku's 500MB compressed slug size limit. Application startup and process management follow Heroku's standard conventions: a Procfile launches Voila bound to the platform-assigned port and all network interfaces, and a .python-version file pins the Python runtime version, following Heroku's current guidance since the legacy runtime.txt mechanism has been deprecated.

Because inference-only PyTorch execution on a ResNet18-based model comfortably fits within a standard Heroku dyno's 512MB memory allocation, no specialized hardware or elevated dyno tier is required for normal operation, though the accompanying deployment documentation explicitly covers the platform's operational failure modes that a real deployment should anticipate: memory-quota exhaustion (Heroku's R14 error), boot-timeout failures if model loading is ever slow enough to exceed the platform's sixty-second startup window (R10), and the practical debugging workflow (`heroku logs --tail`) for diagnosing either. Taken as a whole, Landmark Snap demonstrates that the trained model is not merely a notebook artifact, but a genuinely operable service, deployed under realistic infrastructure constraints and documented well enough for another engineer to redeploy, extend, or hand off.

9. Conclusion

This project set out to answer a specific, practically motivated question: can a convolutional neural network, trained on a modest, curated dataset, reliably recognize landmarks from user-submitted photographs well enough to substitute for missing location metadata in a real product context — and can the resulting model be operationalized, not merely demonstrated in a notebook? The work documented in this report addresses both halves of that question directly.

On the modeling side, two complementary approaches were built, trained, and rigorously compared under a consistent, validation-driven evaluation methodology. The from-scratch convolutional neural network demonstrates that foundational architectural principles — hierarchical feature extraction through channel-doubling convolutional blocks, batch

normalization for training stability, and dropout-based regularization — are sufficient to build a genuinely functioning fifty-class landmark classifier without any pretrained components, meeting the project's baseline accuracy threshold and establishing a transparent, fully-understood point of comparison. The transfer-learning model, built on a frozen, ImageNet-pretrained ResNet18 backbone with a newly trained classification head, was selected specifically because it converts a data-constrained problem into a far more tractable one, and was correspondingly held to, and expected to meet, a materially higher accuracy bar. Both models were exported through a single, consistent TorchScript pipeline that eliminates any gap between how each model was trained and how it is later served, directly connecting the experimentation phase of this project to its deployment phase.

On the systems and product side, the project went beyond a purely academic exercise by shipping "Landmark Snap," a fully deployed, publicly reachable web application that lets any user upload a photograph and receive a live, ranked, visually presented prediction. Getting to that point required solving problems that a notebook-only submission would never surface: cross-version compatibility issues in the interactive upload widget, dependency footprint constraints imposed by the hosting platform's slug size and memory limits, and the operational discipline of documenting startup behavior, memory characteristics, and common failure modes for whoever maintains the deployment next. Alongside deployment, the project's three additional stand-out contributions — disciplined experiment tracking, a content-based image-retrieval capability built on the model's own learned feature space, and an explicit accounting of adjacent product use cases and the model's boundaries — extend the work meaningfully past the minimum bar of "a model that classifies fifty landmark categories."

Taken together, this project demonstrates the complete arc expected of an applied deep learning engineer working on a computer vision problem: framing an ambiguous, real-world business problem in terms a model can solve; building, comparing, and justifying multiple modeling strategies against that framing; validating results honestly using a methodology that resists overfitting to any single evaluation split; and shipping the resulting model as a genuinely usable, documented, and maintainable piece of software rather than leaving it as an isolated experimental artifact. The remaining sections of this report turn to the tangible business value this work creates, the limitations that should be understood before extending its use, and the concrete directions in which this line of work could reasonably continue.

10. Value Statement

The business case for a landmark classification capability rests on a simple observation: location context is one of the highest-leverage signals a photo-sharing or photo-storage platform can attach to an image, and this project demonstrates a viable, low-cost way to recover that context

precisely in the cases where it would otherwise be lost. Several concrete sources of value follow directly from that capability.

The most immediate value is product-facing: automatic, metadata-free location tagging directly enables features users already expect from modern photo platforms, including automatic album organization by trip or location, "photos from this place" surfacing, and smarter search and recommendation. Because this capability activates specifically on images that currently fall through the cracks of metadata-based systems (stripped EXIF data, GPS-less capture devices, re-shared or re-exported images), it closes a real gap in feature coverage rather than duplicating an existing capability, meaning the incremental lift to user experience is concentrated exactly where it is currently missing.

There is also a meaningful cost-avoidance dimension. Manually reviewing or tagging images at the scale a photo-sharing platform operates at is not economically feasible, and even partial human review is expensive and slow relative to the volume of daily uploads. An automated classifier that can be run at inference time on every uploaded image, at a cost measured in milliseconds of CPU time per image rather than human-minutes, converts what would otherwise be an intractable manual workflow into a background process that scales linearly with infrastructure spend rather than headcount.

The image-retrieval stand-out feature adds a second, distinct product surface beyond classification: because the model's learned feature embeddings can be used to find visually similar images independent of class labels, the same underlying model investment supports "more like this" recommendations, near-duplicate detection for storage deduplication, and similarity-based content discovery — capabilities that extend the value of the model well beyond the fifty specific landmark labels it was trained to output, since the feature space itself carries transferable visual information.

There is strategic value as well. A platform that can automatically infer meaningful structure from otherwise unstructured, metadata-free images differentiates itself on data quality and downstream feature richness in a way competitors relying purely on user-supplied or EXIF-derived metadata cannot easily match. This also compounds over time: better-tagged historical data improves the training signal available for future, more capable models, and a validated deployment pipeline (as demonstrated by Landmark Snap) means that improvement cycle — retrain, re-export, redeploy — is already proven to work end to end rather than being a theoretical future capability.

Finally, this project itself demonstrates a template for how such capabilities can be built responsibly and efficiently: using transfer learning rather than training expensive models from scratch, using disciplined experiment tracking rather than ad hoc tuning, and validating

deployability under realistic infrastructure constraints rather than only in a notebook. That template — not just the specific fifty-class model produced here — is itself a transferable organizational asset for tackling the next visual classification problem a photo platform encounters.

11. Limitations

Despite meeting its stated technical objectives, this system has a number of real limitations that should be understood clearly before it is extended toward production use at scale, rather than glossed over in favor of the more flattering results discussed elsewhere in this report.

The most fundamental limitation is the closed-set nature of the classification problem as currently framed. The model can only output one of the fifty landmark classes it was explicitly trained on; it has no built-in mechanism for recognizing that a photograph depicts a landmark, or scene, entirely outside that set. A photo of a landmark not among the fifty trained classes will still be assigned a class label and a (potentially high) confidence score, because softmax normalization guarantees the output probabilities sum to one across only the classes the model knows about — there is no "none of the above" option. In a real deployment, this means the model's raw output cannot be trusted uncritically; it requires either a calibrated confidence threshold below which predictions are suppressed, or a genuinely separate out-of-distribution detection mechanism, neither of which is implemented here.

The dataset itself introduces a second class of limitation. Fifty landmarks, however well-chosen, represent a vanishingly small fraction of the landmarks that exist worldwide, and the training images for any given class are themselves a finite, non-exhaustive sample of how that landmark can appear — under different lighting, weather, seasons, angles, and levels of occlusion by crowds or construction. Performance is correspondingly sensitive to how well a new photo's conditions resemble the training distribution; an landmark photographed at night, in fog, from an unusual angle, or partially obscured is meaningfully more likely to be misclassified than a clear, well-lit, canonically-framed daytime photo, even for a landmark the model was trained on. The dataset is also not perfectly class-balanced, and some landmarks are visually far more distinctive (and therefore easier to learn) than others that share generic architectural features with several other classes, which is likely to produce uneven per-class accuracy that a single aggregate accuracy number obscures.

There are also deployment-level constraints worth naming plainly. Landmark Snap runs inference on CPU-only infrastructure by design, which is adequate for the current single-image, interactive-latency use case but would not scale efficiently to a batch or high-throughput production workload without either GPU-backed infrastructure or a lighter-weight, quantized model variant. The standard Heroku dyno tier used for this deployment also imposes a real memory ceiling that

leaves limited headroom for future model growth (a larger backbone, an ensemble, or additional post-processing) without a corresponding infrastructure upgrade.

Finally, the image-retrieval stand-out feature, while functionally demonstrated, currently searches only a small, randomly sampled gallery of test-set images for latency reasons rather than an exhaustive index, and uses no approximate-nearest-neighbor indexing structure; this is an appropriate simplification for a demonstration but would need to be replaced with a proper vector index (such as FAISS or a managed vector database) before it could support retrieval over a realistically large photo collection.

12. Direction of Future Research

The limitations described above point directly toward a set of concrete, well-motivated directions for extending this work, several of which would meaningfully increase its readiness for production deployment at scale rather than merely improving benchmark accuracy in isolation.

The most impactful near-term direction is addressing the closed-set assumption directly, by adding an explicit open-set recognition or out-of-distribution rejection capability. This could take the form of a calibrated confidence threshold derived from a held-out calibration set, a dedicated "unknown" output category trained against a broad set of non-landmark and off-distribution imagery, or a more principled approach such as energy-based or distance-based out-of-distribution scoring applied to the model's learned feature space — the same feature space already exposed for the image-retrieval stand-out feature could serve double duty here, since anomalously distant embeddings are a natural signal for "this doesn't resemble anything the model was trained on."

A second promising direction is expanding both the breadth and depth of the underlying visual representation. The current transfer-learning model deliberately uses a lightweight, frozen ResNet18 backbone as an appropriate trade-off for this project's scope; a natural next step would be to benchmark newer, larger, or differently structured backbones (EfficientNet variants, Vision Transformers, or ConvNeXt) or to experiment with unfreezing and fine-tuning the later backbone layers rather than only the classification head, which typically improves domain-specific accuracy at the cost of longer training time and greater risk of overfitting on a still-modest dataset. Zero-shot or few-shot approaches built on large vision-language models (such as CLIP) are also worth investigating specifically because they could help address the open-set limitation and the fixed fifty-class ceiling simultaneously, potentially recognizing landmarks well outside the current training set without requiring labeled examples for every new class added.

Dataset expansion is a third clear direction: incorporating additional landmark classes, additional images per existing class (particularly to improve coverage of underrepresented lighting,

seasonal, and angle variation), and potentially additional metadata such as approximate geographic coordinates per class, which would allow the system to move from "which of fifty landmarks is this" toward a richer, hierarchical prediction of country, region, and specific landmark, more closely mirroring how a human would reason about location from an unfamiliar photo.

On the systems side, formalizing the currently manual experiment-tracking discipline into a dedicated platform (MLflow, Weights & Biases, or an equivalent) would preserve the same methodology already established here while adding automatic run versioning, artifact lineage, and multi-run comparison at scale — valuable as the number of experiments, model variants, and contributors grows beyond what a single notebook table can practically track. Similarly, replacing the retrieval feature's small in-memory gallery with a proper approximate-nearest-neighbor vector index would be a prerequisite for retrieval at realistic photo-collection scale. Finally, model compression techniques — quantization, pruning, or knowledge distillation into a smaller student model — would reduce both inference latency and deployment memory footprint, directly addressing the CPU-only, memory-constrained deployment limitations discussed above, and would be a natural precursor to eventually serving this capability at true photo-platform scale or even on-device.

13. References

The technical foundations of this project draw on a small set of well-established works in computer vision and deep learning, alongside the platform and library documentation used to implement, track, and deploy the system described above. Each reference is annotated briefly to clarify how it informed a specific design decision in this project.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). This paper introduces the ResNet architecture and its residual connections, and directly motivates the choice of a ResNet18 backbone for the transfer-learning model described in this report.

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). This paper describes the ImageNet dataset used to pretrain the ResNet18 backbone, and underlies the transfer-learning rationale discussed in the Solution Statement and Architecture sections.

Weyand, T., Araujo, A., Cao, B., & Sim, J. (2020). Google Landmarks Dataset v2 -- A large-scale benchmark for instance-level recognition and retrieval. In Proceedings of the IEEE/CVF

Conference on Computer Vision and Pattern Recognition (CVPR). This paper describes the source dataset from which the fifty-class landmark subset used in this project was curated.

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In Proceedings of the 32nd International Conference on Machine Learning (ICML). This work motivates the batch normalization layers used throughout the from-scratch CNN architecture.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929-1958. This work motivates the dropout regularization used in the classification head of the from-scratch CNN.

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*. The channel-doubling, sequential convolutional-block design pattern used in the from-scratch CNN is directly descended from the design principles established in this work.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In Proceedings of the 3rd International Conference on Learning Representations (ICLR). This paper introduces the Adam optimizer, offered as one of two optimizer choices compared during the experiment-tracking process described in this report.

Radenović, F., Tolias, G., & Chum, O. (2018). Fine-tuning CNN image retrieval with no human annotation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(7), 1655-1668. This work informed the design of the cosine-similarity, feature-embedding-based image retrieval stand-out feature described in this report.

Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*. PyTorch and its torchvision companion library were used for all model implementation, training, and TorchScript export described in this report.

Voila Contributors. (n.d.). Voila documentation. Retrieved from <https://voila.readthedocs.io/>. Voila was used to convert the trained model into the standalone, code-hidden "Landmark Snap" web application described in the deployment section of this report.

Salesforce/Heroku. (n.d.). Heroku Python Support Reference. Heroku Dev Center. Retrieved from <https://devcenter.heroku.com/articles/python-support>. This documentation informed the deployment configuration described in this report, including the use of a .python-version file in

place of the deprecated runtime.txt mechanism and the CPU-only PyTorch dependency strategy adopted to satisfy Heroku's slug size constraints.

Udacity. Landmark Classification & Tagging for Social Media (project specification and starter codebase). This project's dataset subset, notebook structure, and rubric requirements were provided as the foundation upon which the architecture, experimentation, stand-out features, and deployment documented in this report were built.